



TamSang:

ระบบสั่งอาหารที่ร้านอาหาร และจัดการหลังร้าน

จัดทำโดย

นางสาวติชัมพร	ท้วมแก้ว	รหัสนิสิต	67102010161
นายพิชชากร	จลาสุภ	รหัสนิสิต	67102010161
นายอามีน	สาแม	รหัสนิสิต	67102010161

เสนอ

ผู้ช่วยศาสตราจารย์ ดร.วราภรณ์ วิทยานนท์

รายงานนี้เป็นส่วนหนึ่งรายวิชา CP242 Database Systems

มหาวิทยาลัยศรีนครินทรวิโรฒ

สารบัญ

	หน้า
Problem Statement	1
Conceptual Design	3
Logical Design	4
Normalization	11

TamSang: ระบบสั่งอาหารที่ร้านอาหาร และจัดการหลังร้าน

คือ ระบบสำหรับใช้ในการสั่งอาหารโดยการสแกนผ่าน QR Code และช่วยให้เจ้าของร้านสามารถจัดการเมนูและราคาได้แบบ Real-time ซึ่งช่วยลดวงจรค่าใช้จ่ายในการพิมพ์เมนูใหม่ซ้ำๆไปได้ และสรุปวิเคราะห์ยอดขายและจัดการข้อมูลทำให้เจ้าของร้านไม่ต้องคาดเดารูทกิจอีกต่อไป แต่สามารถตัดสินใจทุกอย่างบนพื้นฐานของข้อมูลจริงได้

ข้อมูลหลักที่ต้องจัดเก็บ

- ข้อมูลเมนูและวัตถุดิบ:

รายละเอียดเมนู: ชื่อเมนู, คำอธิบาย, ราคา, มีส่วนประกอบอะไรบ้าง, ระดับความเผ็ด(ต้องการให้มีตัวเลือกระดับความเผ็ดในเมนูนี้ไหม), สามารถเพิ่มท็อปปิ้งอะไรได้บ้าง, รูปภาพ และสถานะการขาย(พร้อมขาย, หมด)

หมวดหมู่: จัดกลุ่มเมนูและกลุ่มวัตถุดิบเพื่อความสะดวกในการจัดการ

วัตถุดิบ: รายการวัตถุดิบทั้งหมดที่ใช้ในร้านและสถานะความพร้อม

- ข้อมูลการขายและการให้บริการ:

ออเดอร์: ข้อมูลการสั่งอาหาร, เวลาที่สั่ง, สถานะการทำรายการ, และรายการอาหารในแต่ละออเดอร์

รายการอาหารในออเดอร์: ข้อมูลเจาะจงของแต่ละจาน เช่น ระดับความเผ็ด, ความต้องการพิเศษ, จำนวน และราคา

ท็อปปิ้งในรายการอาหาร: ข้อมูลท็อปปิ้งที่ลูกค้าเลือกเพิ่มในแต่ละจาน

ข้อมูลโต๊ะ: เลขโต๊ะ, จำนวนที่นั่ง, และสถานะการใช้งาน

- ข้อมูลสมาชิก(พนักงาน) และการเงิน:

ข้อมูลสมาชิก: ชื่อ-นามสกุล, ข้อมูลบัญชีผู้ใช้, และตำแหน่ง

ข้อมูลการชำระเงิน: ยอดเงิน, วิธีการชำระเงิน, วันที่ชำระ

ข้อมูลต้นทุน: บันทึกรายจ่ายรายวัน, รายละเอียดค่าใช้จ่าย และผู้รับผิดชอบ

- ข้อมูลระบบและบันทึกการทำงาน:

การตั้งค่าร้าน: ข้อมูลพื้นฐานของร้านค้าที่จำเป็นสำหรับระบบ

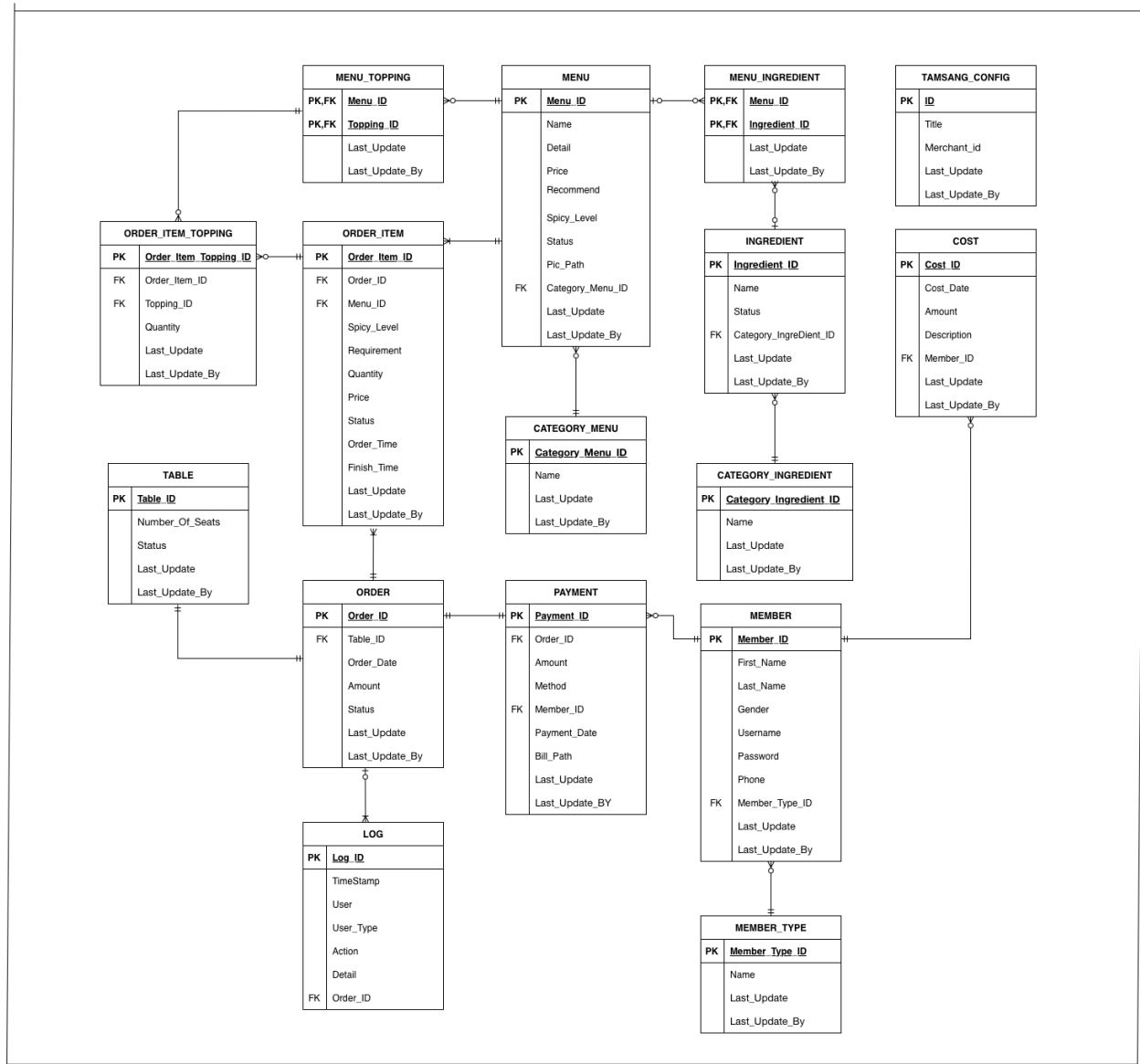
บันทึกเหตุการณ์: ประวัติการทำรายการต่างๆ ในระบบ, วันเวลา, ผู้ทำรายการ, และรายละเอียดการเปลี่ยนแปลง

ผู้ใช้งานระบบ มีอยู่ 4 กลุ่มดังนี้:

1. ลูกค้า: สแกนสั่งอาหาร
2. พนักงาน: เปิดโต๊ะเพื่อให้สามารถใช้ QR Code สั่งอาหารได้ และกด approve ว่าลูกค้าได้ชำระเงินเรียบร้อยแล้ว
3. เชฟ: จัดรายการอาหารที่ลูกค้าสั่งเข้ามา จัดการวัตถุดิบว่าหมดหรือพร้อมใช้
4. เจ้าของร้าน: จัดการเมนู และดูสรุปภาพรวมต่างๆ

Conceptual Design

ER Diagram



Logical Design

Relational Schema

1. หมวดเมนูและวัตถุดิบ

```
CATEGORY_MENU (  
    Category_Menu_ID PK,  
    Name,  
    Last_Update,  
    Last_Update_By  
)
```

```
MENU (  
    Menu_ID PK,  
    Name,  
    Detail,  
    Price,  
    Recommend,  
    Spicy_Level,  
    Status,  
    Pic_Path,  
    Category_Menu_ID FK -> CATEGORY_MENU(Category_Menu_ID),  
    Last_Update,  
    Last_Update_By  
)
```

```
CATEGORY_INGREDIENT (  
    Category_Ingredient_ID PK,  
    Name,  
    Last_Update,  
    Last_Update_By  
)
```

```
INGREDIENT (  
    Ingredient_ID PK,  
    Name,  
    Status,  
    Category_Ingredient_ID FK -> CATEGORY_INGREDIENT(Category_Ingredient_ID),  
    Last_Update,  
    Last_Update_By  
)
```

```
MENU_INGREDIENT (  
    Menu_ID PK, FK -> MENU(Menu_ID),  
    Ingredient_ID PK, FK -> INGREDIENT(Ingredient_ID),  
    Last_Update,  
    Last_Update_By  
)
```

```

MENU_TOPPING (
    Menu_ID PK, FK -> MENU(Menu_ID),
    Topping_ID PK, FK -> MENU(Menu_ID),
    Last_Update,
    Last_Update_By
)

```

2. หมวดการขายและการให้บริการ

```

TABLE (
    Table_ID PK,
    Number_Of_Seats,
    Status,
    Last_Update,
    Last_Update_By
)

```

```

ORDER (
    Order_ID PK,
    Table_ID FK -> TABLE(Table_ID),
    Order_Date,
    Amount,
    Status,
    Last_Update,
    Last_Update_By
)

```

```
ORDER_ITEM (  
    Order_Item_ID PK,  
    Order_ID FK -> ORDER(Order_ID),  
    Menu_ID FK -> MENU(Menu_ID),  
    Spicy_Level,  
    Requirement,  
    Quantity,  
    Price,  
    Status,  
    Order_Time,  
    Finish_Time,  
    Last_Update,  
    Last_Update_By  
)
```

```
ORDER_ITEM_TOPPING (  
    Order_Item_Topping_ID PK,  
    Order_Item_ID FK -> ORDER_ITEM(Order_Item_ID),  
    Topping_ID FK -> MENU(Menu_ID),  
    Quantity,  
    Last_Update,  
    Last_Update_By  
)
```

3. หมวดสมาชิก(พนักงาน) และการเงิน

```
MEMBER_TYPE (  
    Member_Type_ID PK,  
    Name,  
    Last_Update,  
    Last_Update_By  
)
```

```
MEMBER (  
    Member_ID PK,  
    First_Name,  
    Last_Name,  
    Gender,  
    Username,  
    Password,  
    Phone,  
    Member_Type_ID FK -> MEMBER_TYPE(Member_Type_ID),  
    Last_Update,  
    Last_Update_By  
)
```

PAYMENT (

Payment_ID PK,

Order_ID FK -> ORDER(Order_ID),

Amount,

Method,

Member_ID FK -> MEMBER(Member_ID),

Payment_Date,

Bill_Path,

Last_Update,

Last_Update_By

)

COST (

Cost_ID PK,

Cost_Date,

Amount,

Description,

Member_ID FK -> MEMBER(Member_ID),

Last_Update,

Last_Update_By

)

4. หมวดยระบบและบันทึกการทำงาน

```
TAMSANG_CONFIG (  
    ID PK,  
    Title,  
    Merchant_id,  
    Last_Update,  
    Last_Update_By  
)  
  
LOG (  
    Log_ID PK,  
    TimeStamp,  
    User,  
    User_Type,  
    Action,  
    Detail,  
    Order_ID PK, FK -> ORDER(Order_ID)  
)
```

Normalization

1NF: ผ่าน เพราะ ไม่มีการเก็บข้อมูลซ้ำแบบ multivalued โดยส่วนที่มีโอกาสเกิดขึ้นคือ

Phone ใน relation MEMBER ป้องกันที่ส่วนของ business rule ให้สามารถกรอกเบอร์โทรศัพท์ได้แค่เบอร์เดียว

และ ส่วนของ topping ใน Order จึงแก้ปัญหาด้วยการสร้าง relation ORDER_ITEM_TOPPING ขึ้นมาเก็บ topping ในแต่ละ order_item แทนที่จะเก็บเป็น String (เช่น "ไข่ดาว, ไข่เจียว, ไข่ต้ม") ในคอลัมน์เดียวแทน

2NF: ผ่าน เพราะ ได้ทำการแยกตารางให้เป็นแบบ Full Functional Dependency

เช่น ตาราง ORDER_ITEM_TOPPING

Key: (Order_Item_Topping_ID, Topping_ID)

FD: Order_Item_Topping_ID, Topping_ID -> Quantity

ตาราง ORDER_ITEM

Key: (Order_Item_ID)

FD: Order_Item_ID -> Requirement

FD: Order_Item_ID -> Spicy_Level

3NF: ได้ทำการแยกทุกตาราง ไม่มี non-prime attribute ขึ้นกับ non-prime attribute

เช่น

CATEGORY_MENU (Category_Menu_ID PK, Name)

MENU (Menu_ID PK, Name, Category_Menu_ID FK ->

CATEGORY_MENU(Category_Menu_ID))